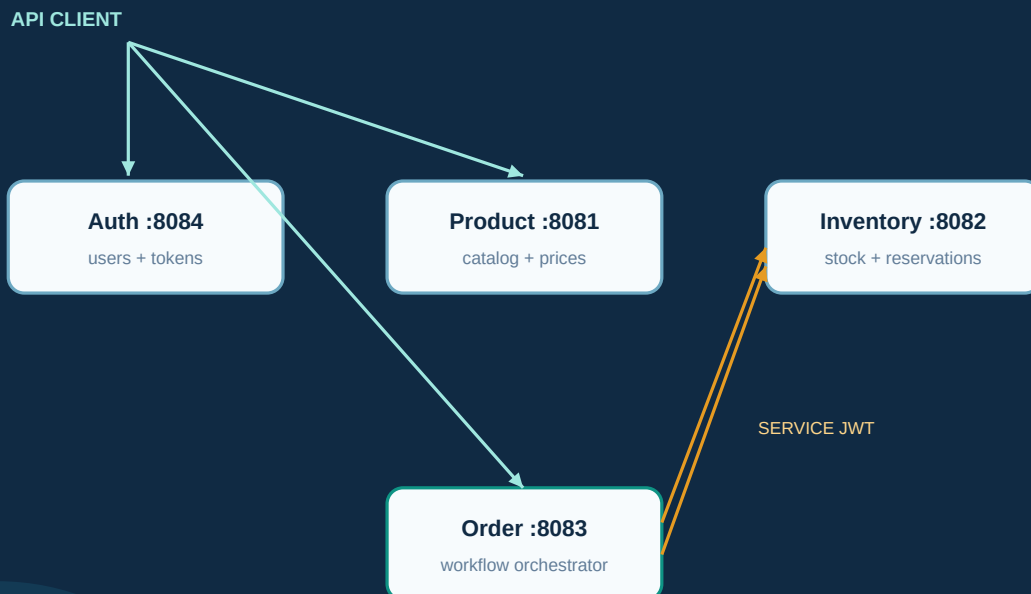


E-commerce microservices POC

A beginner's guide to how the whole application works

Java 21 | Spring Boot 4.1.1 | JWT RBAC

H2 | Resilience4j | Docker Compose



Start here: what you are looking at

This repository is a learning-oriented e-commerce backend built from four separately running Spring Boot applications and one shared Java library. A client can register, log in, browse products, place an order, cancel it, and - with an ADMIN role - manage users, products, and inventory.

The one-sentence mental model

Auth proves who you are. Product owns what can be sold and at what price. Inventory owns how much can be reserved. Order coordinates the other services and records the sale.

What this guide will teach you

- How a request moves from HTTP through security, controller, service, repository, and database.
- Why each service owns a separate database and communicates through APIs instead of JPA relationships.
- How USER, ADMIN, and internal SERVICE roles are represented in JWTs and enforced.
- How order creation uses synchronous calls, local transactions, and compensation instead of one distributed transaction.
- How row locks, optimistic versions, timeouts, retries, circuit breakers, and bulkheads address different failure modes.
- How to start the stack, exercise it manually, trace one request, and migrate from H2 to MySQL.

Scope and honest boundaries

The implementation follows strong proof-of-concept conventions: clear service ownership, stateless bearer authentication, consistent Problem Details errors, safe logging, bounded downstream calls, and Docker health checks. It is not a production platform yet. Its databases are in memory, signing uses one shared HMAC secret, and there is no gateway, message broker, refresh-token system, durable tracing backend, or automated test suite.

POC means proof of concept

A POC proves an architectural idea and creates a runnable learning surface. It should expose tradeoffs clearly; it should not pretend that development defaults are production controls.

Suggested reading path

- 1 Read Parts I and II once to build the system map.
- 2 Run the stack while reading Part III so the security flow becomes concrete.
- 3 Follow one order through Part IV and search its correlation ID in the logs.
- 4 Use Parts V and VI when debugging or planning production changes.
- 5 Keep the API and glossary appendices open while working in the code.

Contents

Start here: what you are looking at	2
1. Microservices without the buzzwords	5
2. Architecture at a glance	6
3. Repository and package map	7
4. What happens to every HTTP request	8
5. Data ownership and the four databases	9
6. Auth service: registration and login	10
7. JWT anatomy and local validation	11
8. Role-based access control	12
9. Internal SERVICE tokens	13
10. Product service	14
11. Inventory service	15
12. Is multithreading used?	16
13. Order service: the orchestrator	17
14. Place-order flow, step by step	18
15. Saga compensation and cancellation	19
16. Resilience4j fault tolerance	20
17. Failure classification	21
18. Global exception handling	22
19. Correlation IDs and AOP logging	23
20. Configuration and Docker Compose	24
21. Start and stop the application	25
22. Manual end-to-end test	26
23. Fault-tolerance exercise	27
24. H2 consoles and development safety	28
25. Moving each service to MySQL	29
26. Do we need Eureka?	30

27. Production-readiness roadmap	31
28. Troubleshooting guide	32
A. Auth and product API reference	33
B. Inventory and order API reference	34
C. Code compass	35
D. Beginner glossary	36
E. Your first week in this codebase	37

PART I

Build the mental model

Learn what is a service, what is only a library, and how the running pieces connect.

1. Microservices without the buzzwords

A microservice is an independently running application that owns a focused business capability. In this project, each service has its own Spring Boot main class, HTTP port, security filter chain, configuration, JPA repositories, and database. The services happen to live in one Gradle repository, but they still run as separate JVM processes or containers.

Term	Meaning in this repository	Common beginner mistake
Service	A separately running Spring Boot application.	Calling every Java package a microservice.
Module	A Gradle build unit. Four modules are services; platform-common is a library.	Assuming every module has an HTTP port.
Database owner	Only one service reads or writes that service's tables.	Joining tables across service databases.
API contract	HTTP path plus request, response, status, and security rules.	Treating controller code as the whole contract.
Orchestrator	Order service coordinates product and inventory calls.	Confusing orchestration with a distributed transaction.

A monolith and this POC solve boundaries differently

Question	Typical monolith	This microservices POC
Deployment	One application artifact.	Four bootable JARs or containers.
Data	One database can join all tables.	Four independent H2 databases.
Calls	In-process Java method calls.	HTTP between order, product, and inventory.
Failure	Most code fails together.	A dependency can fail while other services remain healthy.
Consistency	One local database transaction is often enough.	Cross-service work needs compensation and operational repair.

The goal is not maximum service count

A useful boundary has a reason: independent ownership, scaling, release cadence, security, or domain responsibility. Splitting code without a boundary adds network failure without gaining autonomy.

2. Architecture at a glance

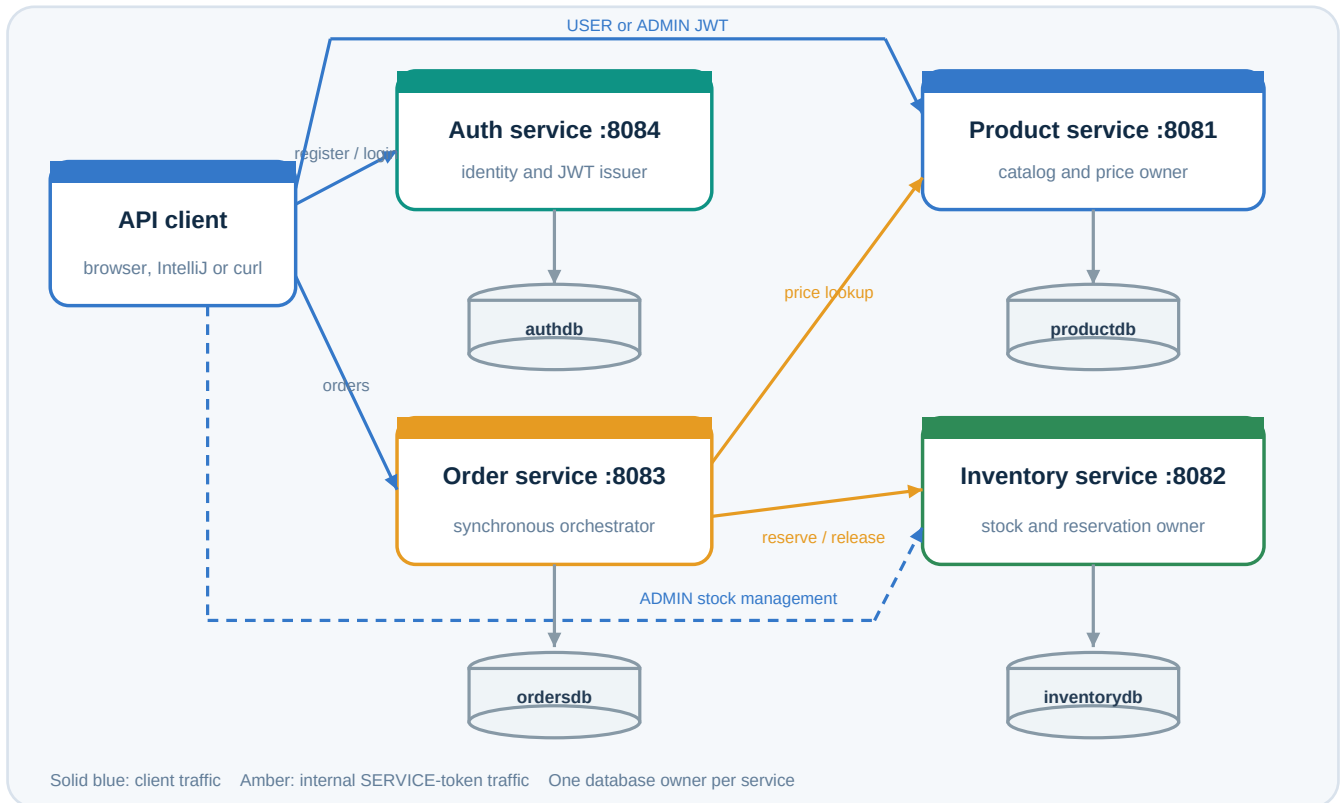


Figure 1. Four runtime services, four owned databases, and one internal orchestration path.

Module	Port	Owns	Calls
platform-common	-	Shared infrastructure only; no business data.	Nothing at runtime.
auth-service	8084	Users, password hashes, roles, enabled state.	No business service.
product-service	8081	Products, active state, authoritative price.	No business service.
inventory-service	8082	Available and reserved quantity by SKU.	No business service.
order-service	8083	Orders, captured prices, status, ownership.	Product and inventory.

What is deliberately absent

- No API gateway: clients address each service port directly.
- No Eureka server: fixed URLs and Docker Compose DNS are enough for this topology.
- No message broker: order orchestration is synchronous and blocking.
- No shared business database: a SKU is a string contract, not a cross-database foreign key.
- No call from product or inventory back to order: order is the only business-call initiator.
- No auth lookup on every request: resource services validate self-contained JWTs locally.

Important consequence

Auth can be temporarily unavailable after a user has a valid token, and other services can still validate it. The tradeoff is that role or enabled-state changes do not revoke an already issued token immediately.

3. Repository and package map

The root is a Gradle multi-project build. The root build defines Java 21 and shared repositories. Each service module declares its own Spring Boot dependencies. The common module is a Java library registered as Spring Boot auto-configuration.

```
ecom-microservices/
  build.gradle      root Java 21 and plugin versions
  settings.gradle   lists the five modules
  compose.yaml      local multi-container topology
  Dockerfile        shared multi-stage image build
  .env.example      safe configuration template
  requests.http     complete manual workflow
  platform-common/  shared infrastructure library
  auth-service/     users and tokens
  product-service/  catalog and prices
  inventory-service/ stock and reservations
  order-service/    order workflow and resilience
```

The package pattern inside a service

Package	Responsibility	Should not do
controller	Maps HTTP methods, paths, status codes, principals, and DTOs.	Own business transactions.
models.request	Deserializes and validates client input.	Expose JPA entities.
models.response	Defines stable JSON returned to callers.	Leak passwords or persistence internals.
service	Implements business rules and transaction boundaries.	Depend on servlet details.
repository	Loads and stores entities through Spring Data JPA.	Call remote services.
domain	Represents persisted state and domain behavior.	Become a shared cross-service entity.
config	Security, clients, seeds, and typed properties.	Contain request-specific decisions.
exception / error	Names business failure conditions.	Serialize itself directly.
client	Order-only adapters for downstream HTTP APIs.	Forward a caller bearer token.

Why request and response models are separate

A request is what the caller is allowed to choose. A response is what the service promises to reveal. For example, order creation accepts SKU and quantity only; customer identity and price are server-owned.

Code compass

```
settings.gradle
```

```
platform-common/src/main/resources/META-INF/spring/...AutoConfiguration.imports
```

```
/src/main/java/com/ecommerce/.../
```

PART II

Follow requests and data

See the standard Spring request pipeline, transaction boundaries, and owned schemas.

4. What happens to every HTTP request

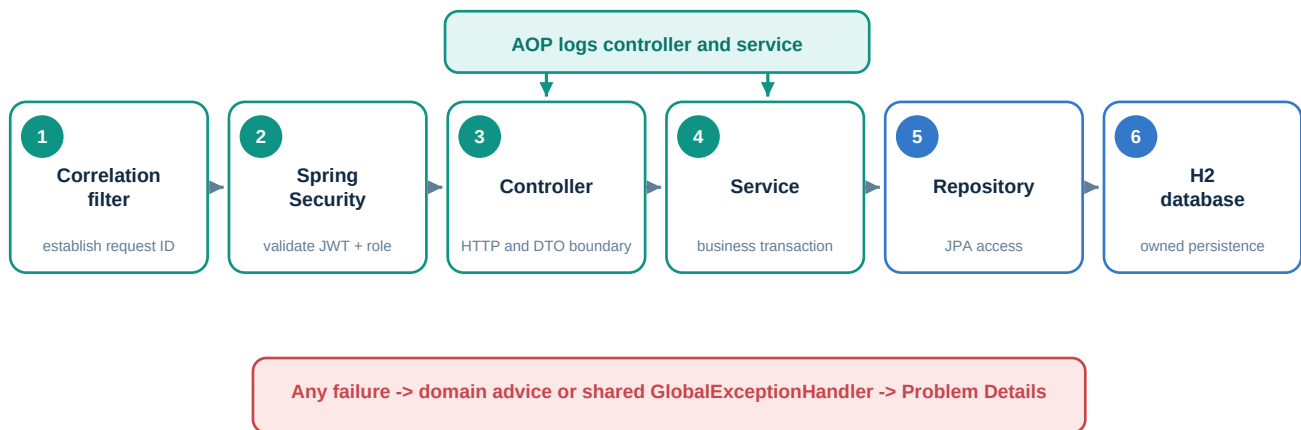


Figure 2. The common request path. Domain exceptions and framework failures share one safe wire format.

- 1 **CorrelationIdFilter** accepts a safe X-Correlation-ID or creates a UUID, echoes it, and places it in the logging MDC.
- 2 **Spring Security** checks whether the route is public; otherwise it verifies JWT signature, issuer, audience, expiry, and role.
- 3 **Controller** converts HTTP input into a validated request record and obtains the authenticated principal.
- 4 **Service** applies business rules inside a local transaction, often annotated with `@Transactional`.
- 5 **Repository** maps Java entities to the service's own H2 tables through Spring Data JPA.
- 6 **Response model** is serialized as JSON, or an exception handler creates application/problem+json.

Where AOP fits

The shared `LoggingAspect` wraps calls that enter classes annotated with `@RestController` or `@Service`. It records start, completion or failure, duration, method, path, principal, role authorities, actor, and correlation ID. It does not log arguments, request bodies, return values, passwords, or tokens.

Proxy rule to remember

Spring AOP observes calls entering a proxied bean. One method calling another method on the same object normally bypasses that proxy. This matters when you later add annotations that depend on AOP.

5. Data ownership and the four databases

Every service has its own in-memory H2 database. There are no cross-database foreign keys. Matching values such as SKU and customer email are copied into the owning service's records. That duplication is intentional: it prevents one service from reaching into another service's schema.

Database	Tables / entity	Important fields and rules
authdb	app_users / AppUser	Unique normalized email; BCrypt hash; USER or ADMIN; enabled; audit timestamps; @Version.
productdb	products / Product	Unique uppercase SKU; name; optional description; decimal price; active.
inventorydb	inventory_items / InventoryItem	Immutable unique SKU; available; reserved; @Version; row lock for mutation.
ordersdb	purchase_orders + order_items	Customer email; CONFIRMED/CANCELLED; captured price; totals; timestamps; local parent-child relation.

Why prices are copied into an order

Product service is authoritative at checkout time, but an existing order must remain historically stable. OrderItem stores unitPrice and lineTotal. If an administrator changes a catalog price tomorrow, yesterday's order total remains unchanged.

What H2 MySQL mode does - and does not do

Every JDBC URL uses MODE=MySQL to make development SQL behavior closer to MySQL. It does not make H2 identical to MySQL. Collation, lock timing, indexing, timestamp precision, decimal behavior, and query plans still require validation against a real MySQL instance.

In-memory means disposable

Data lives only while that service JVM is alive. Stopping product service destroys productdb but does not destroy the other three databases. This can intentionally create inconsistent demo state after partial restarts.

PART III

Identity, JWT, and RBAC

Understand login, self-contained bearer tokens, route authorization, and the internal machine identity.

6. Auth service: registration and login

Registration

POST /api/auth/register is public. It trims and lowercases the email, trims the display name, hashes the password with BCrypt, and always creates an enabled USER. A caller cannot register as ADMIN.

Field	Validation
email	Required, valid email, maximum 320 characters.
displayName	Required, maximum 100 characters.
password	12-72 characters with uppercase, lowercase, and a digit.

```
POST http://localhost:8084/api/auth/register
Content-Type: application/json
```

```
{
  "email": "new.user@example.com",
  "displayName": "New User",
  "password": "NewUserPass123!"
}
```

Login

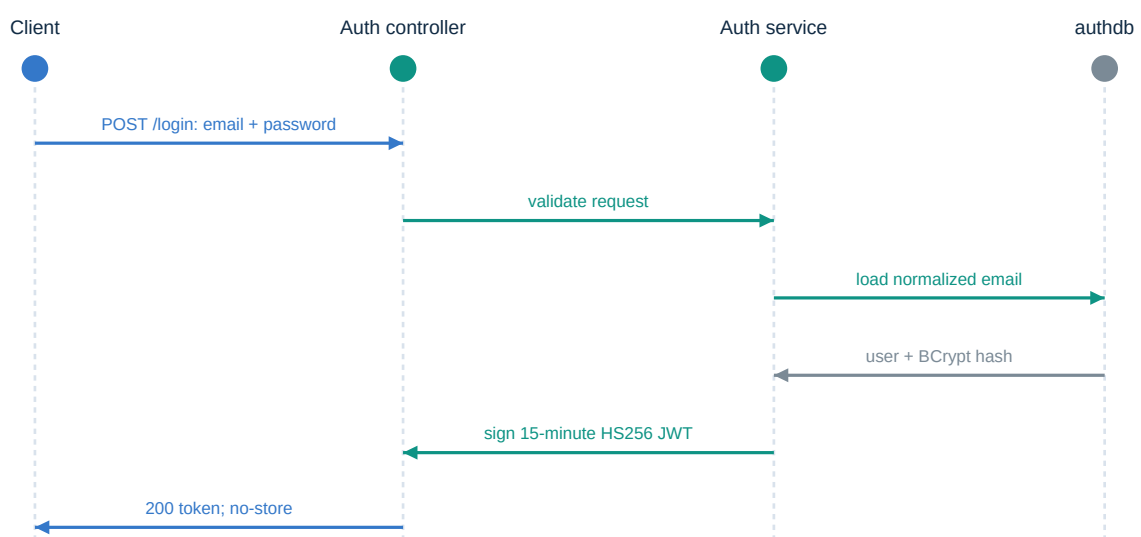


Figure 3. Login checks a stored BCrypt hash and returns a signed 15-minute access token.

Spring Security's DaoAuthenticationProvider compares the submitted password to the stored BCrypt hash. Disabled users fail the same way as wrong credentials, reducing account-enumeration clues. The response sets Cache-Control:

no-store and Pragma: no-cache.

```
POST http://localhost:8084/api/auth/login
Content-Type: application/json

{"email":"user@example.com","password":"ChangeMeUser123!"}

200 OK
{
  "accessToken": "<JWT omitted>",
  "tokenType": "Bearer",
  "expiresAt": "2026-09-01T12:15:00Z",
  "expiresInSeconds": 900
}
```

Code compass

auth-service/.../controller/AuthController.java

auth-service/.../service/AuthService.java

auth-service/.../service/UserService.java

7. JWT anatomy and local validation

JWT stands for JSON Web Token. It carries signed claims. The services trust the claims only after validating the HS256 signature, issuer, audience, and standard timestamp rules. They do not call auth-service to validate each request.

Claim	User token value	Why it exists
iss	ecommerce-auth by default	Names the issuer.
aud	ecommerce-api by default	Limits the intended audience.
sub	Database user ID as text	Stable subject identifier.
email	Normalized email	Order ownership identity in this POC.
userId	Database user ID as number	Convenient typed identity.
roles	USER or ADMIN	Converted to ROLE_USER or ROLE_ADMIN.
iat / exp	Issue and expiry times	15-minute default lifetime.
jti	Random UUID	Unique token identifier.

Why the same secret must be configured everywhere

Auth-service signs HS256 tokens, and every resource service validates them using the same JWT_SECRET bytes. The secret must be at least 32 UTF-8 bytes. Issuer and audience must also match, or validation fails.

Production security gap

A shared HMAC secret means every validating service also has enough key material to mint tokens. A production design normally uses asymmetric signing: the identity provider keeps the private key and resource services receive only public keys through JWKS, with key rotation.

Stateless does not mean instantly revocable

A token carries its role and expiry. If an admin disables a user, an already issued token may continue working against product, inventory, or order until it expires. The default exposure window is at most 15 minutes. Sensitive auth admin operations recheck the acting admin in authdb, but other services do not introspect centrally.

8. Role-based access control

Capability	Public	USER	ADMIN	SERVICE
Register and login	Yes	-	-	-
Read own profile	-	Yes	Yes	-
Manage users	-	-	Yes	-
Browse catalog	-	Yes	Yes	SKU lookup only
Create, update, delete product	-	-	Yes	-
Read or set inventory	-	-	Yes	-
Reserve or release inventory	-	-	-	Yes
Create and manage own orders	-	Yes	Yes	-
View or cancel any order	-	-	Yes	-

Authentication versus authorization

Question	Failure	Result
Who are you?	Token missing, invalid, expired, wrong issuer or audience.	401 authentication_required.
May you do this?	Valid token, but required role is absent.	403 access_denied.
Do you own this order?	Valid USER asks for another customer's order.	404 order_not_found to avoid disclosure.

Admin safety rules

- An administrator cannot demote, disable, or delete their own account.
- The final enabled administrator cannot be demoted, disabled, or deleted.
- Database write locks protect those invariants when admin requests arrive concurrently.
- Registration always creates USER; only ADMIN can change stored roles.

Defense in depth

Admin URLs are protected in the Spring Security filter chain and AdminUserController also uses @PreAuthorize. UserService then enforces domain rules using current database state.

9. Internal SERVICE tokens

When order-service calls product or inventory, it does not forward the human caller's bearer token. A RestClient interceptor creates a new machine token valid for 60 seconds.

Claim	Internal value	Meaning
sub	order-service	The machine principal.
roles	[SERVICE]	A narrow internal authority.
actorSub	Original human token subject	Who initiated the workflow for audit logs.
iat / exp	60-second window	Short-lived synchronous credential.
iss / aud / jti	Shared configured values and UUID	Same validation contract and unique ID.

- 1 USER or ADMIN token arrives at order-service and is validated.
- 2 Order controller derives the customer identity from email, falling back to sub.
- 3 Before each downstream request, DownstreamAuthenticationInterceptor requires an authenticated JWT.
- 4 ServiceTokenProvider signs a 60-second token with ROLE_SERVICE.
- 5 The interceptor adds that token and propagates X-Correlation-ID.
- 6 Product accepts SERVICE only for SKU lookup; inventory accepts SERVICE only for reserve and release.

Least privilege in practice

Even though the SERVICE token is valid in both resource services, route rules prevent it from listing all products, editing products, reading inventory, or setting stock. It gets only the operations needed by the order workflow.

Code compass

```
order-service/.../config/ServiceTokenProvider.java
order-service/.../config/DownstreamAuthenticationInterceptor.java
platform-common/.../config/PlatformCommonAutoConfiguration.java
```

PART IV

Business services and order workflow

Walk through catalog management, stock reservations, order creation, cancellation, and consistency.

10. Product service

Product service owns the catalog and is the only authority for price and active state. A client may browse with USER or ADMIN. Only ADMIN can create, replace, or delete products. Order-service uses the narrower SERVICE-only SKU lookup path.

Method and path	Role	Behavior
GET /api/products	USER, ADMIN	List every product.
GET /api/products/{id}	USER, ADMIN	Get by numeric ID.
GET /api/products/sku/{sku}	USER, ADMIN, SERVICE	Get by normalized SKU.
POST /api/products	ADMIN	Create; 201 plus Location.
PUT /api/products/{id}	ADMIN	Replace all editable fields.
DELETE /api/products/{id}	ADMIN	Delete; 204.

Product contract

Request field	Rules	Response
sku	1-64; letter/number first; then letters, numbers, dot, underscore, hyphen.	Normalized uppercase.
name	Required; maximum 200.	id, sku, name.
description	Optional; maximum 2000.	description.
price	Positive; at most 17 integer digits and 2 decimals.	decimal price.
active	Boolean.	active.

The service checks SKU uniqueness before saving, and the database unique constraint remains authoritative if concurrent create or update requests race. A data-integrity race becomes a controlled 409 conflict.

Cross-service deletion is not cascading

Deleting a product does not delete inventory and must never rewrite historical order items. Each service owns its own lifecycle. Operational cleanup must be explicit.

11. Inventory service

Inventory separates quantity that is available for new orders from quantity already reserved by confirmed orders. Administrators manage records and available stock; only order-service can reserve or release.

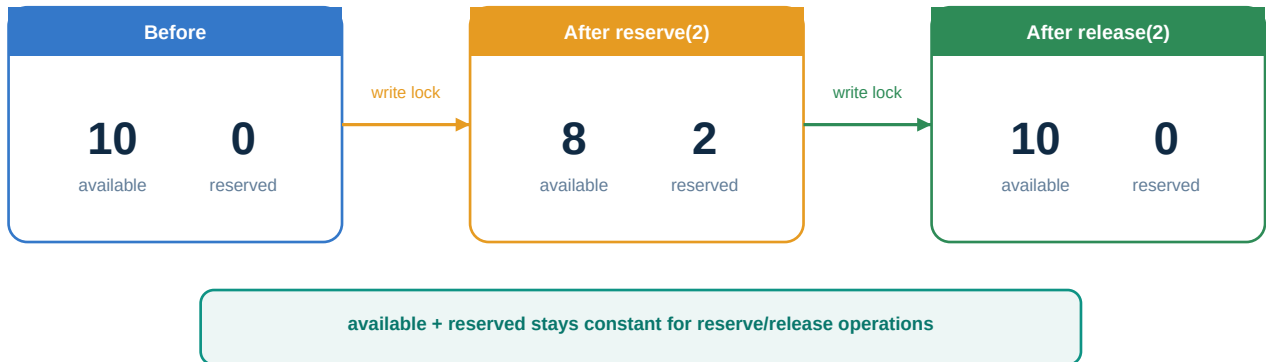


Figure 4. Reservation moves quantity between two buckets under a row lock; release reverses it.

Method and path	Role	Behavior
GET /api/inventory	ADMIN	List stock records.
GET /api/inventory/{sku}	ADMIN	Get one SKU.
POST /api/inventory	ADMIN	Create SKU with available quantity.
PUT /api/inventory/{sku}	ADMIN	Replace available quantity only.
POST /api/inventory/{sku}/reservations	SERVICE	Move available to reserved.
DELETE /api/inventory/{sku}/reservations?quantity=N	SERVICE	Move reserved back to available.

Concurrency protection

`InventoryRepository.findBySkuForUpdate` uses a `PESSIMISTIC_WRITE` lock. If two request threads try to reserve the same SKU, one waits for the other's local database transaction. The second sees the latest quantity before deciding whether stock is sufficient. `@Version` adds optimistic change detection and is returned in the response.

Available replacement nuance

`PUT` changes `availableQuantity` only. It does not erase `reservedQuantity`. This lets an administrator replenish or correct sellable stock without silently losing existing reservations.

12. Is multithreading used?

Short answer

Yes, concurrent HTTP requests are handled by servlet-container worker threads. No, the application does not create its own threads or asynchronous business workflow.

Mechanism	Present?	What it means
Servlet request threads	Yes	Multiple API calls can execute at the same time.
Manual Thread / Executor	No	The code does not manage a custom pool.
@Async / CompletableFuture	No	Business work remains on the request thread.
Parallel streams	No	SKU work is intentionally sequential.
Blocking RestClient	Yes	The order request thread waits for product and inventory.
Semaphore bulkhead	Yes	At most 10 concurrent calls per downstream dependency; no new threads.
Database locks	Yes	Conflicting mutations serialize at the database row.

Why thread safety still matters

Spring singleton controllers and services are shared between request threads. They stay safe because they do not store per-request mutable fields. Request identity lives in Spring Security's context and correlation ID in MDC. Both are thread-local. If you later introduce asynchronous work, you must propagate that context deliberately.

Where concurrency rules live

- Inventory mutation: row lock by SKU prevents overselling during competing reservations.
- Order cancellation: row lock by order prevents releasing the same reservation twice.
- Admin management: locks protect the final-enabled-admin invariant.
- Product creation: unique database constraint resolves concurrent duplicate SKU races.
- Product update: there is no @Version, so concurrent updates can be last-write-wins.

13. Order service: the orchestrator

Order service is the only service that performs a multi-service business workflow. It owns customer-visible orders, but it does not own product validity, prices, or stock. Those facts are requested from their owners.

Method and path	Caller	Behavior
POST /api/orders	USER, ADMIN	Resolve prices, reserve stock, persist CONFIRMED order; 201.
GET /api/orders	USER own / ADMIN all	List visible orders newest first.
GET /api/orders/{id}	Owner or ADMIN	Read one visible order.
POST /api/orders/{id}/cancel	Owner or ADMIN	Release stock and mark CANCELLED.

The create request is deliberately small

```
POST http://localhost:8083/api/orders
Authorization: Bearer <user-token>
Content-Type: application/json
X-Correlation-ID: manual-order-001

{
  "items": [
    {"sku": "SKU-LAPTOP", "quantity": 1},
    {"sku": "SKU-HEADPHONES", "quantity": 2}
  ]
}
```

The caller cannot submit customerEmail, role, unitPrice, lineTotal, total, or status. Identity comes from the validated JWT. Price and active state come from product-service. Status begins as CONFIRMED only after all reservations succeed.

Before any side effect

OrderService normalizes SKUs, merges duplicate lines with Math.addExact, and fetches every unique product price before reserving anything. This reduces unnecessary compensation when a later product is missing or inactive.

14. Place-order flow, step by step

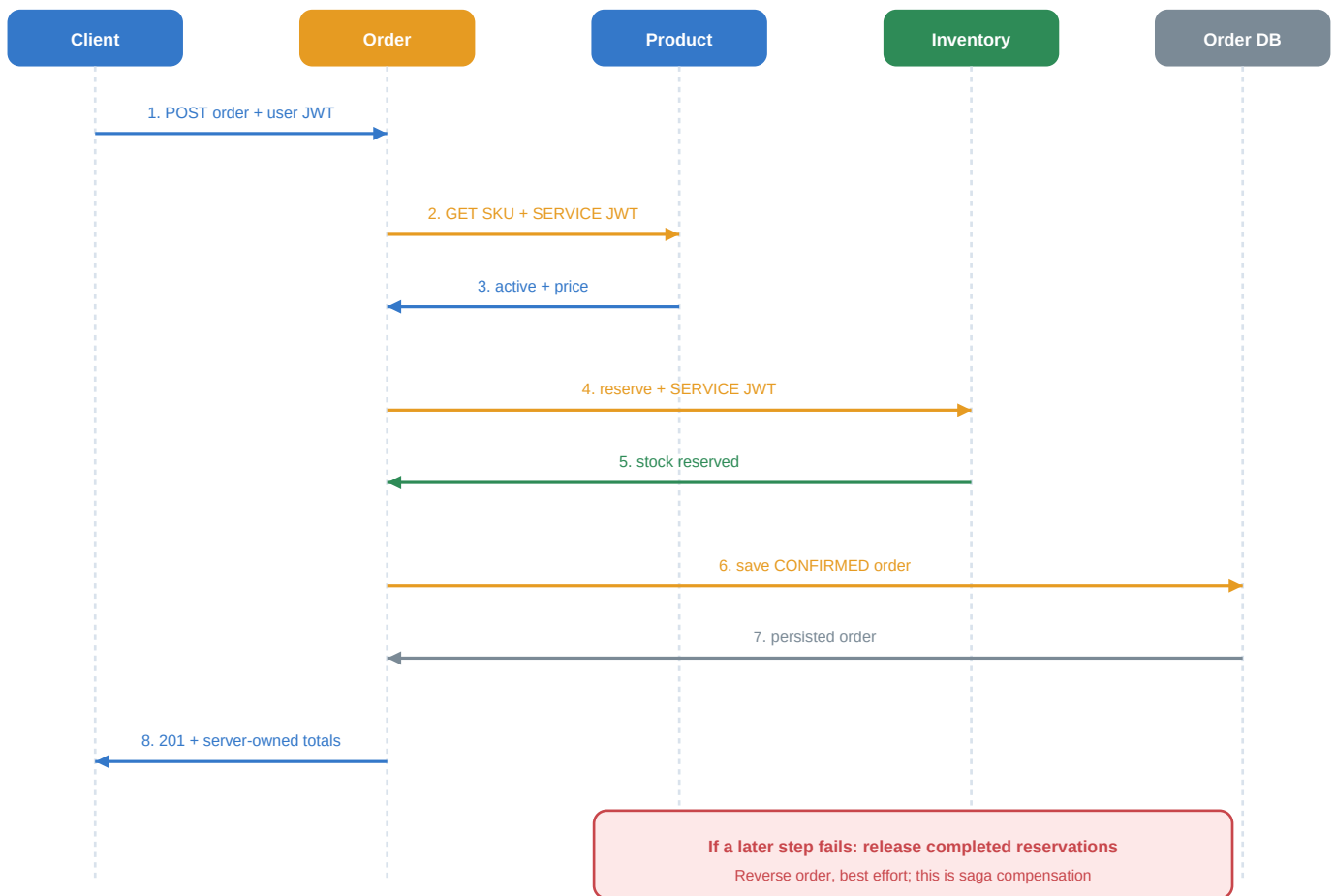


Figure 5. The happy path uses user authentication at the edge and SERVICE authentication downstream.

- 1 Spring Security validates the USER or ADMIN JWT locally in order-service.
- 2 OrderController chooses lowercased email as customer identity, falling back to subject.
- 3 OrderService normalizes and aggregates duplicate SKU lines.
- 4 CatalogClient looks up every SKU. The product must exist, be active, and contain a valid price.
- 5 InventoryClient reserves each unique SKU sequentially. Inventory locks each SKU row.
- 6 Only after all reservations succeed does order-service create PurchaseOrder and OrderItem entities.
- 7 Each item captures its unit price, calculates lineTotal, and contributes to total with two-decimal rounding.
- 8 saveAndFlush writes the CONFIRMED order; the controller returns 201 and Location `/api/orders/{id}`.

Local @Transactional is not global

The annotation protects the order database transaction only. It cannot roll back HTTP requests already committed by inventory-service. That is why compensation exists.

15. Saga compensation and cancellation

Creation compensation

OrderService records each completed reservation in memory. If a later reservation or order persistence fails, it releases completed reservations in reverse order. If a release also fails, that failure is attached as a suppressed exception; the original business failure remains primary.

Cancellation

- 1 Owner or ADMIN requests POST /api/orders/{id}/cancel.
- 2 Order repository loads the order with a PESSIMISTIC_WRITE lock; USER lookup also matches customer identity.
- 3 If status is already CANCELLED, the existing response is returned without another release.
- 4 Order-service releases every item through inventory-service using a SERVICE token.
- 5 If a release fails midway, completed releases are re-reserved on a best-effort basis.
- 6 After all releases succeed, status becomes CANCELLED and is flushed.
- 7 If saving CANCELLED fails, order-service attempts to restore all reservations.

Property	What the POC guarantees	What it cannot guarantee
Order creation	Prices resolved first; successful reservations compensated on later failure.	Atomic commit across HTTP and order DB.
Cancellation	Same-order callers serialize; second cancel does not release twice.	Perfect recovery from ambiguous network outcomes.
Compensation	Reverse-order, best-effort repair.	Guaranteed completion after process crash.

Production evolution

Add idempotency keys, reservation IDs, a transactional outbox, durable events, reconciliation jobs, and tracing. Compensation should become observable state, not only a best-effort loop in one request.

PART V

Failures, errors, and observability

Learn how the application bounds downstream failure and gives operators one traceable error contract.

16. Resilience4j fault tolerance

Only order-service has downstream business clients, so resilience policies live there. Timeout, retry, circuit breaker, and bulkhead solve different problems and must be combined deliberately.

Pattern	Catalog lookup	Inventory mutation	Reason
Connect timeout	500 ms	500 ms	Do not wait indefinitely to establish a connection.
Read timeout	2 s	2 s	Bound response waiting time.
Retry	3 attempts; 200 ms exponential backoff	None	Reads are safer to retry; a timed-out mutation may have committed.
Circuit breaker	Yes	Yes	Fail fast when a dependency is unhealthy.
Semaphore bulkhead	10 calls; no wait	10 calls; no wait	Limit concurrent pressure without creating threads.

Circuit-breaker settings

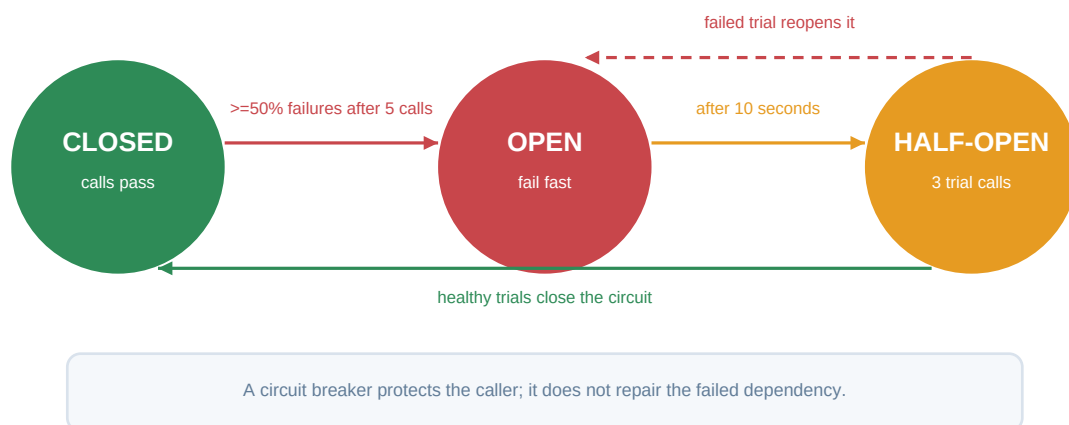


Figure 6. Count-based circuit state. Business rejections are ignored; infrastructure failures count.

Operational thresholds

Setting	Value
Sliding window	10 calls, count based.
Minimum before evaluation	5 calls.
Failure-rate threshold	50 percent.
Slow call	At least 1.5 seconds; threshold 50 percent.
Open duration	10 seconds.
Half-open trials	3 calls; automatic transition.

Why inventory has no automatic retry

If a reservation times out after inventory committed it, order-service cannot know whether it changed stock. Blindly retrying could reserve twice. Safe retries require idempotency keys or reservation identifiers.

17. Failure classification

Situation	HTTP	Stable code	Interpretation
Missing or inactive product	422	product_unavailable	Business input cannot be fulfilled.
Inventory rejects reservation	409	inventory_reservation_failed	Business state conflict.
Network or unexpected downstream HTTP	502	downstream_service_failure	Bad gateway to a dependency.
Circuit open or bulkhead full	503	downstream_capacity_exhausted	Temporary capacity or health rejection.
Bad request DTO	400	validation_failed	Client input violates constraints.
Missing or invalid JWT	401	authentication_required	Authentication failed.
Wrong role	403	access_denied	Authenticated but forbidden.

Product-unavailable and inventory business exceptions are excluded from circuit-breaker failure counting. A customer asking for unavailable stock should not convince the system that the inventory server is technically down.

A retry is not a universal fix

Retries add traffic and latency. Retry only when an operation is safe and a transient failure is plausible. Never hide persistent failure behind an unbounded retry loop.

18. Global exception handling

platform-common registers a shared RFC 9457 error layer. Each service adds a higher-priority domain advice for its business exceptions. The result is one predictable application/problem+json shape.

```
HTTP/1.1 400 Bad Request
Content-Type: application/problem+json
X-Correlation-ID: manual-validation-001

{
  "type": "urn:ecommerce:problem:validation_failed",
  "title": "Validation failed",
  "status": 400,
  "detail": "items: must not be empty",
  "instance": "/api/orders",
  "code": "validation_failed",
  "timestamp": "2026-09-01T12:00:00Z",
  "correlationId": "manual-validation-001",
  "errors": ["items: must not be empty"]
}
```

What is standardized

- Bean validation, malformed JSON, missing values, wrong types, missing resources, and unsupported methods or media types.
- Authentication and authorization failures, including WWW-Authenticate on 401.
- Domain errors such as duplicate email, duplicate SKU, insufficient stock, and order dependency failure.
- Unexpected exceptions: a generic client detail is returned while the full stack trace is logged server-side.

Why stable code matters

HTTP status is broad. The code field is the application-specific value clients and dashboards can branch on. The human-readable detail may evolve without breaking that contract.

Code compass

```
platform-common/.../problem/ProblemDetailsFactory.java
platform-common/.../problem/GlobalExceptionHandler.java
/.../controller/*ExceptionHandler.java
```

19. Correlation IDs and AOP logging

A correlation ID lets one business request be found across several log streams. The filter accepts a caller value only when it is 1-64 safe characters; otherwise it creates a UUID. It echoes the ID in the response and clears MDC in finally so one request ID cannot leak into the next worker-thread task.

- 1 Client sends X-Correlation-ID: manual-order-001 to order-service.
- 2 CorrelationIdFilter stores it as a request attribute and in SLF4J MDC.
- 3 LoggingAspect includes it in controller and service operation logs.
- 4 DownstreamAuthenticationInterceptor forwards it to product and inventory.
- 5 Their filters and AOP logs use the same value.
- 6 The client receives the value in every response header and Problem Details body.

```
docker compose logs order-service product-service inventory-service |  
  Select-String 'manual-order-001'
```

Logged	Intentionally not logged
Operation start / completion / failure	Bearer token
Duration and exception type	Password or password hash
HTTP method and path	Method arguments and request body
Principal, roles, actorSub	Return value or response body
Service name and correlation ID	Secrets

Current observability level

Actuator exposes health and selected Resilience4j state, but there is no Prometheus registry, log aggregation, distributed tracing backend, dashboard, or alerting system yet.

PART VI

Run, test, and evolve it

Start the stack safely, exercise a complete workflow, diagnose common failures, and plan production changes.

20. Configuration and Docker Compose

Docker Compose builds the same multi-stage Dockerfile four times using `SERVICE_NAME`. The build stage uses a Java 21 JDK; the runtime stage uses a smaller JRE, runs as non-root UID 10001, and includes curl for health checks.

Environment setup

```
Copy-Item .env.example .env

# Edit .env and set a new secret of at least 32 UTF-8 bytes:
JWT_SECRET=<new-random-secret>
JWT_ISSUER=ecommerce-auth
JWT_AUDIENCE=ecommerce-api
```

Do not expose the actual .env secret

.env is gitignored and must remain local. All four services need the same JWT settings. Compose rejects an empty `JWT_SECRET` before containers start.

Container	Host port	Health	Startup dependency
auth-service	8084	/actuator/health	None.
product-service	8081	/actuator/health	None.
inventory-service	8082	/actuator/health	None.
order-service	8083	/actuator/health	Waits for healthy product and inventory.

Compose gives every container a DNS name matching its service name. Therefore order-service uses `http://product-service:8081` and `http://inventory-service:8082` inside the Docker network. When started directly, its defaults are `localhost:8081` and `localhost:8082`.

21. Start and stop the application

Recommended: Docker Compose

```
docker compose up --build --wait
docker compose ps
docker compose logs -f
```

Check all four services:

```
curl http://localhost:8081/actuator/health
curl http://localhost:8082/actuator/health
curl http://localhost:8083/actuator/health
curl http://localhost:8084/actuator/health
```

```
docker compose down
```

Stopping loses data

There are no database containers or persistent volumes. `docker compose down` stops JVMs, so all four H2 databases disappear.

Direct Gradle development

```
.\gradlew.bat clean bootJar --no-daemon

# In four terminals, export the same JWT settings, then run:
.\gradlew.bat :auth-service:bootRun
.\gradlew.bat :product-service:bootRun
.\gradlew.bat :inventory-service:bootRun
.\gradlew.bat :order-service:bootRun
```

A shell does not automatically import `.env` for Gradle. Export `JWT_SECRET`, `JWT_ISSUER`, and `JWT_AUDIENCE` in every terminal. Auth seeding defaults off in direct startup; Compose explicitly enables it. Product and inventory seeding default on.

22. Manual end-to-end test

Open `requests.http` in IntelliJ IDEA and run it from top to bottom. Named responses store `userToken`, `adminToken`, and created IDs for later requests. Because H2 is disposable, restarting the stack gives you a clean run.

- 1 Log in as the seeded USER and ADMIN; both should return 200 bearer tokens.
- 2 Call product list anonymously; expect 401 Problem Details.
- 3 Call product list as USER; expect 200 and seeded products.
- 4 Attempt product creation as USER; expect 403. Repeat as ADMIN; expect 201.
- 5 Attempt direct inventory access as USER; expect 403. Manage stock as ADMIN.
- 6 Place an order as USER using SKU and quantity only; expect 201 and JWT-derived customer identity.
- 7 List as USER and confirm only owned orders appear; list as ADMIN and see all orders.
- 8 Send an empty items array; expect 400 with validation errors and `correlationId`.
- 9 Cancel as owner or ADMIN; inventory is released and status becomes CANCELLED.
- 10 Cancel again; expect the existing cancelled order without another release.

Seeded local data

SKU	Product	Price	Initial stock
SKU-LAPTOP	Pro 14 Laptop	1299.00	10
SKU-PHONE	128 GB Smartphone	699.00	25
SKU-HEADPHONES	Noise-cancelling Headphones	249.00	50

Role	Email	Local-only password
ADMIN	admin@example.com	ChangeMeAdmin123!
USER	user@example.com	ChangeMeUser123!

Repeatability note

Registration and create operations may return 409 when repeated without a reset. That is correct uniqueness behavior, not a test failure.

23. Fault-tolerance exercise

- 1 Start the full stack and obtain an ADMIN token.
- 2 Stop the catalog: `docker compose stop product-service`.
- 3 Submit valid order requests repeatedly. Early dependency failures return 502.
- 4 After enough counted failures, the catalog circuit opens and new attempts fail fast with 503.
- 5 Inspect GET `http://localhost:8083/actuator/circuitbreakers` using the ADMIN token.
- 6 Restart the catalog: `docker compose start product-service`.
- 7 Wait at least 10 seconds. Successful half-open trial calls allow the circuit to close.

```
GET http://localhost:8083/actuator/circuitbreakers
Authorization: Bearer <admin-token>

docker compose stop product-service
docker compose start product-service
```

Demo side effect

Stopping product-service destroys productdb. On restart, only the predefined product seed is recreated. Any product you created manually is gone, while other services keep their own state until they stop.

24. H2 consoles and development safety

Service	Console URL	JDBC URL	Default
Auth	http://localhost:8084/h2-console	jdbc:h2:mem:authdb	Disabled
Product	http://localhost:8081/h2-console	jdbc:h2:mem:productdb	Enabled
Inventory	http://localhost:8082/h2-console	jdbc:h2:mem:inventorydb	Enabled
Order	http://localhost:8083/h2-console	jdbc:h2:mem:ordersdb	Enabled

Use username sa and a blank password for local inspection. The security configurations permit the console paths, and frame options allow same-origin rendering. These are development conveniences only.

Never expose these consoles

Before any shared or deployed environment, disable H2 consoles, remove blank database credentials, and use network controls plus authenticated administration tooling.

Actuator

- Every service exposes health and info.
- Order also exposes circuit breaker, retry, and bulkhead state and event endpoints.
- Order health is public; its other exposed Actuator endpoints require ADMIN.
- Product and inventory health/info are public. Auth health is public; other exposed auth endpoints require ADMIN.

25. Moving each service to MySQL

The code uses standard JPA mappings and externalized datasource settings, so the migration is mainly dependency, configuration, schema, and operational work. Keep one schema and least-privilege credential set per service.

- 1 Add `com.mysql:mysql-connector-j` as a runtime dependency to all four persistence services.
- 2 Provision `auth`, `product`, `inventory`, and `order` schemas independently.
- 3 Set `AUTH_DB_URL/USERNAME/PASSWORD` and the corresponding `PRODUCT`, `INVENTORY`, and `ORDER` variables.
- 4 Introduce Flyway or Liquibase versioned migrations.
- 5 Replace `ddl-auto=update` with `ddl-auto=validate` after migrations run.
- 6 Disable H2 consoles and development seeds.
- 7 Configure Hikari connection pools, TLS, secret delivery, lock timeouts, transaction isolation, and backups.
- 8 Run concurrency, migration, and load checks against real MySQL rather than assuming H2 compatibility.

Verify in MySQL	Why
Email and SKU collation	Case sensitivity affects unique constraints.
InnoDB pessimistic locks	Reservation and cancellation correctness depends on row-lock behavior.
Decimal precision	Prices and totals must remain exact.
UTC timestamp precision	Audit and ordering assumptions must hold.
Indexes and query plans	H2 performance is not representative.
Pool size versus servlet threads	Blocked connections can become the real bulkhead.

26. Do we need Eureka?

Current answer: no

There is one known instance of each service. Local startup uses externalized localhost URLs, and Docker Compose provides service-name DNS. Eureka would add another runtime component without solving a present requirement.

Deployment model	Recommended discovery
This Docker Compose POC	Compose DNS and configured base URLs.
Kubernetes	Kubernetes Services and cluster DNS in most cases.
Managed container platform	Native service discovery or internal load balancers.
Dynamic VM-based Spring fleet	Eureka or Consul may be appropriate.

Discovery is not a gateway

Service discovery answers: where is a service instance? An API gateway answers: how do external clients enter, route, authenticate, rate-limit, and observe APIs? They are separate decisions. Add either only when the deployment and operational requirements justify it.

27. Production-readiness roadmap

Area	POC today	Production direction
Data	In-memory H2; ddl-auto update.	MySQL per service, migrations, backups, tested recovery.
Identity	Shared HS256 secret.	OIDC provider, asymmetric keys/JWKS, rotation, machine credentials.
Sessions	15-minute access token only.	Refresh/revocation strategy, MFA, verification, throttling.
Consistency	In-request best-effort compensation.	Idempotency, reservation IDs, outbox/events, reconciliation.
Edge	Four direct client ports.	Gateway/load balancer, TLS, CORS, rate limits, API versioning.
Observability	Logs, correlation ID, Actuator.	Metrics, traces, aggregation, dashboards, alerts, SLOs.
Deployment	Compose, no restart/resource rules.	Orchestrator, probes, limits, autoscaling, rollout policy.
Quality	Manual requests.http workflow.	Unit, integration, contract, security, concurrency, and chaos tests.

A sensible evolution order

- 1 Move secrets and identity to production-grade infrastructure.
- 2 Move persistence to MySQL with migrations and backups.
- 3 Add automated tests around domain, security, contracts, and concurrency.
- 4 Add idempotency and durable consistency for order workflows.
- 5 Add centralized metrics, logs, traces, dashboards, and alerting.
- 6 Choose gateway, discovery, and asynchronous messaging based on real deployment and scale needs.

Industry standard is contextual

Good engineering is not a checklist of fashionable components. It is explicit ownership, bounded failure, secure defaults, testable contracts, observable behavior, and infrastructure appropriate to the deployment.

28. Troubleshooting guide

Symptom	Likely cause	First action
Compose says JWT_SECRET is missing	.env absent, empty, or not in project root.	Copy .env.example to .env and set 32+ random bytes.
401 on a protected route	Missing/expired token or mismatched secret/issuer/audience.	Log in again; compare JWT settings across services.
403 with a valid token	Role does not match the endpoint.	Check the RBAC matrix and roles claim.
Order returns 422	Product is absent or inactive.	Check product by SKU with an allowed token.
Order returns 409	Inventory rejected reservation.	Inspect ADMIN inventory quantity.
Order returns 502	Product/inventory HTTP or network failure.	Check dependency health and order logs.
Order returns 503	Circuit open or bulkhead full.	Inspect order Actuator resilience endpoints.
Manual request gets duplicate conflict	A create step was rerun against existing H2 state.	Use returned IDs, choose new values, or restart cleanly.
New product vanished after fault demo	Product JVM restart destroyed productdb.	Recreate it or use a seeded SKU.
Gradle services reject each other's tokens	Different terminal environment values.	Export the exact same JWT settings everywhere.

A disciplined debugging loop

- 1 Capture status, stable error code, and correlationId from the response.
- 2 Search logs by correlationId across the services involved.
- 3 Identify whether the failure is authentication, authorization, validation, domain state, capacity, or dependency health.
- 4 Inspect the owning service's database or Actuator state, not another service's tables.
- 5 Reproduce with the smallest request and record which local state must be reset.

APPENDIX

API reference and glossary

Keep these compact references beside you while exploring controllers and requests.http.

A. Auth and product API reference

Auth service - localhost:8084

Method	Path	Caller	Success	Request / response
POST	/api/auth/register	Public	201	RegisterRequest -> UserResponse
POST	/api/auth/login	Public	200	LoginRequest -> TokenResponse
GET	/api/users/me	USER, ADMIN	200	- -> UserResponse
GET	/api/admin/users?page=&size;=	ADMIN	200	- -> UserPageResponse
PATCH	/api/admin/users/{id}/role	ADMIN	200	UpdateRoleRequest -> UserResponse
PATCH	/api/admin/users/{id}/enabled	ADMIN	200	UpdateEnabledRequest -> UserResponse
DELETE	/api/admin/users/{id}	ADMIN	204	No body

Product service - localhost:8081

Method	Path	Caller	Success	Request / response
GET	/api/products	USER, ADMIN	200	- -> ProductResponse list
GET	/api/products/{id}	USER, ADMIN	200	- -> ProductResponse
GET	/api/products/sku/{sku}	USER, ADMIN, SERVICE	200	- -> ProductResponse
POST	/api/products	ADMIN	201	ProductUpsertRequest -> ProductResponse
PUT	/api/products/{id}	ADMIN	200	ProductUpsertRequest -> ProductResponse
DELETE	/api/products/{id}	ADMIN	204	No body

B. Inventory and order API reference

Inventory service - localhost:8082

Method	Path	Caller	Success	Request / response
GET	/api/inventory	ADMIN	200	- -> InventoryResponse list
GET	/api/inventory/{sku}	ADMIN	200	- -> InventoryResponse
POST	/api/inventory	ADMIN	201	CreateInventoryRequest -> response
PUT	/api/inventory/{sku}	ADMIN	200	InventoryStockUpdateRequest -> response
POST	/api/inventory/{sku}/reservations	SERVICE	200	ReservationRequest -> response
DELETE	/api/inventory/{sku}/reservations?quantity=N	SERVICE	200	- -> InventoryResponse

Order service - localhost:8083

Method	Path	Caller	Success	Request / response
POST	/api/orders	USER, ADMIN	201	CreateOrderRequest -> OrderResponse
GET	/api/orders	USER own / ADMIN all	200	- -> OrderResponse list
GET	/api/orders/{id}	Owner or ADMIN	200	- -> OrderResponse
POST	/api/orders/{id}/cancel	Owner or ADMIN	200	- -> OrderResponse

Response record shapes

- **UserResponse:** id, email, displayName, role, enabled, createdAt, updatedAt.
- **ProductResponse:** id, sku, name, description, price, active.
- **InventoryResponse:** id, sku, availableQuantity, reservedQuantity, version.
- **OrderResponse:** id, customerEmail, status, total, createdAt, updatedAt, items.
- **Order item:** id, sku, quantity, unitPrice, lineTotal.

C. Code compass

Concern	Primary implementation
Shared auto-configuration	platform-common/.../config/PlatformCommonAutoConfiguration.java
JWT settings and validation	platform-common/.../security/JwtSecurityProperties.java and auto-configuration
Problem Details	platform-common/.../problem/ProblemDetailsFactory.java; GlobalExceptionHandler.java
Correlation	platform-common/.../web/CorrelationIdFilter.java
AOP logs	platform-common/.../logging/LoggingAspect.java
Registration and admin rules	auth-service/.../service/UserService.java
User token issuing	auth-service/.../service/AuthService.java
Catalog rules	product-service/.../service/ProductService.java
Stock locks and rules	inventory-service/.../service/InventoryService.java; InventoryRepository.java
Order saga	order-service/.../service/OrderService.java
Downstream clients	order-service/.../client/CatalogClient.java; InventoryClient.java
Machine token	order-service/.../config/ServiceTokenProvider.java
Resilience settings	order-service/src/main/resources/application.yml
Runtime topology	compose.yaml; Dockerfile; .env.example
Manual workflow	requests.http

Best entry point for reading code

Start at a controller method, follow its request record into the service, then inspect repository and entity. For order creation, also follow CatalogClient, InventoryClient, and the downstream interceptor.

D. Beginner glossary

Term	Plain-language meaning
Actuator	Spring Boot operational endpoints such as health and circuit-breaker state.
AOP	Aspect-oriented programming: proxy-based behavior that wraps selected method calls.
Authentication	Proving who the caller is.
Authorization	Deciding what an authenticated caller may do.
Bulkhead	A concurrency limit that prevents one dependency from consuming all caller capacity.
Circuit breaker	A state machine that fails fast when a dependency is repeatedly unhealthy.
Compensation	A business action that tries to undo an earlier cross-service side effect.
Controller	The HTTP boundary that maps methods, paths, bodies, status, and principal.
Correlation ID	A request identifier repeated in responses and logs across services.
DTO / model	A request or response data shape used at an API boundary.
Entity	A Java object mapped to a database table by JPA.
Idempotent	Repeating an operation has the same effect as doing it once.
JWT	A signed token carrying claims such as subject, role, issuer, and expiry.
Local transaction	An atomic unit of work inside one service database.
MDC	Thread-local logging context used here for correlationId.
Orchestrator	A service that explicitly coordinates calls to other services.
Pessimistic lock	A database lock that makes conflicting writers wait.
Problem Details	The RFC 9457 standard JSON structure for HTTP errors.
RBAC	Role-based access control: permissions assigned through USER, ADMIN, SERVICE.
Repository	A Spring Data interface that loads and stores entities.
Retry	Repeating a safe operation after a likely transient failure.
Saga	A multi-service workflow using local transactions plus compensation.
Service discovery	Finding the current network location of a service instance.
SKU	Stock keeping unit: the shared product/inventory identifier.

E. Your first week in this codebase

- 1 **Day 1:** start Compose, run requests.http, and trace manual-order-001 across logs.
- 2 **Day 2:** read each controller and security configuration; compare the RBAC matrix to actual matchers.
- 3 **Day 3:** follow ProductService and InventoryService down to JPA entities and repository locks.
- 4 **Day 4:** step through OrderService.create and cancel; write down every local commit and compensation.
- 5 **Day 5:** stop dependencies, inspect Resilience4j states, and classify each response by failure type.
- 6 **Next:** choose one production gap - migrations, asymmetric JWT, idempotency, or observability - and design it before coding.

You now have the map. The fastest way to deepen it is to follow one real request from controller to database and across every network boundary, using the correlation ID as your thread.

Document basis: the checked-in repository implementation and configuration at 1 September 2026. Commands target Windows PowerShell unless otherwise stated. The actual local .env secret was neither read nor included.